Lecture 16: Memory Hierarchy III — Associativity, Eviction & Performance

Session 19 · Mon Nov 2, 2026

Recap: Lecture 15

A cache has S sets of W lines. Each line holds a valid bit, a tag, and one block of B bytes. The cache splits every address into three fields: the **set index** picks the set, the **tag** says which block a line holds, and the **offset** picks the byte inside the block.

In a **direct-mapped** cache each set has one line, so two blocks with the same index evict each other. Misses come in three kinds: **compulsory** (the first read of a block), **capacity** (the working set is larger than the cache) and **conflict** (blocks in use compete for the same set while other sets have room).

Addresses that differ by a large power of two have the same index. Summing a 4096 × 4096 `int` array by columns got 4 times faster when each row was made 64 bytes longer, because the column's blocks then spread over all the sets.

Today. Sets with more than one line, how a cache chooses which block to evict, what happens on a write, and how to measure and predict cache performance.

Outline

What you will learn today:

1. How a set-associative cache finds a block among the lines of a set
2. How two lines per set remove the conflict miss from the tiny cache traced in Lecture 15
3. Four ways to choose which block to evict, and which one real caches imitate
4. A measurement that counts the lines per set of a real L1 cache
5. What a cache does on a write: write-through, write-back and the dirty bit
6. How to compute the average time of a memory access
7. Why the order of the three loops in matrix multiplication changes its speed, predicted by hand and checked with **`perf`**
8. How caches on different cores keep their copies the same, and how that can slow down two threads that share no data
9. How FlashAttention speeds up a transformer by never writing a large matrix to memory

Set-Associative Caches

A **W-way set-associative** cache has W lines in each set. A block can go into any line of the set its index selects. To read an address, the cache:

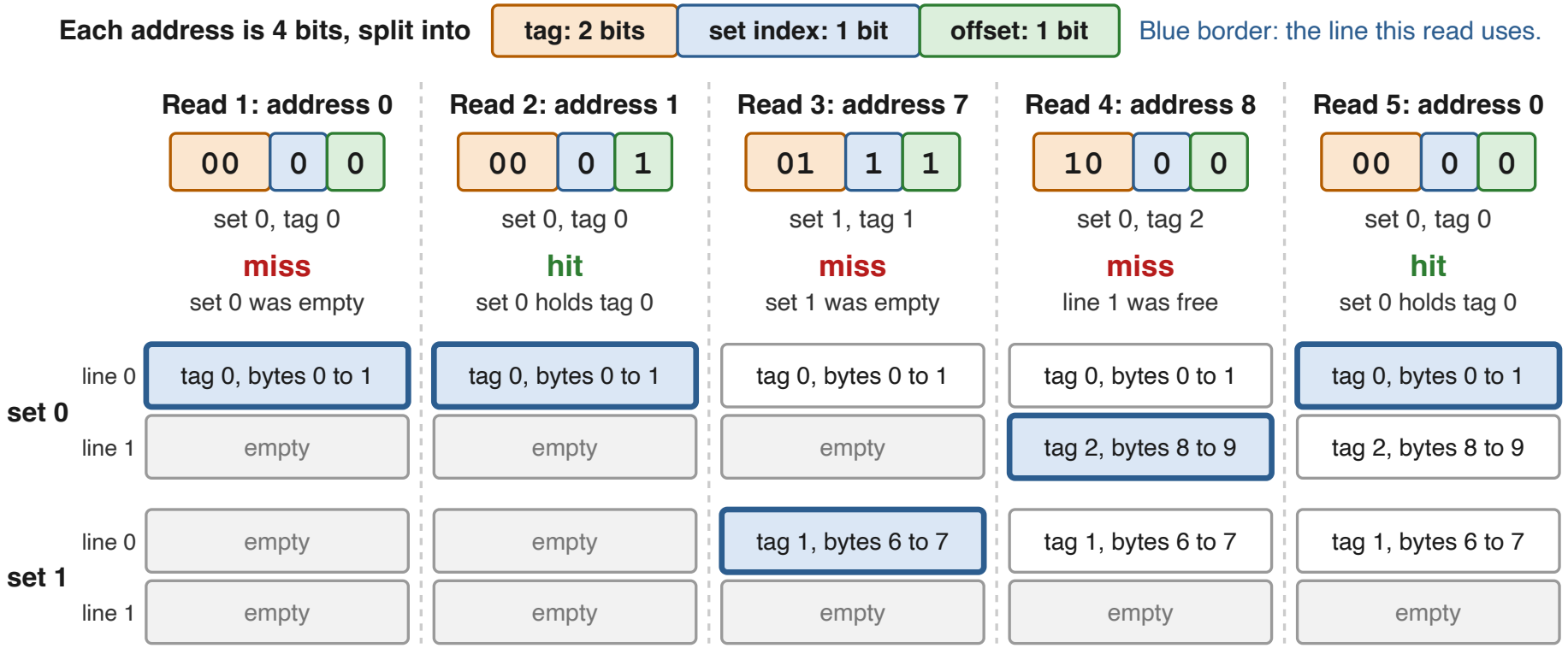
1. picks the set from the index bits, as before;
2. compares the address's tag with the tags of **all W lines** of that set at the same time. If one matches and its valid bit is 1, it is a hit;
3. on a miss, fetches the block and puts it in an empty line of the set. If every line is full, it must choose one block to evict.

Direct-mapped is the case W = 1. A **fully associative** cache has one set that holds every line, so any block can go anywhere. For an 8 KB cache with 64-byte blocks, which has 128 lines in all:

Organization	W (lines per set)	S (sets)	Index bits	Tag bits (64-bit address)
direct-mapped	1	128	7	51
4-way set-associative	4	32	5	53
fully associative	128	1	0	58

Tracing the Tiny Cache With Two Lines per Set

Keep the same 8 bytes of data as the tiny direct-mapped cache, but arrange them as 2 sets of 2 lines, with 2-byte blocks. Now b = 1, s = 1 and t = 2. The program reads addresses 0, 1, 7, 8, 0 again:



The direct-mapped cache had 1 hit and 4 misses. This one has 2 hits and 3 misses, all compulsory.
Blocks 0 and 8 can now be in set 0 at the same time, so the last read hits.

Eviction Policies

When a set is full and a new block must come in, the cache must choose one block to evict. Four common rules:

Policy	Evicts	Strength	Weakness
LRU (least recently used)	the block that was used longest ago	fits temporal locality: recently used blocks stay	the cache must track the order of use
FIFO (first in, first out)	the block that was loaded longest ago	simple: one counter per set	evicts a block that is still in use
Random	any block, chosen at random	no bookkeeping at all	sometimes evicts a block about to be used
LFU (least frequently used)	the block used the fewest times	keeps blocks that are used very often	an old block that was popular stays too long

Example: one set with 3 lines reads blocks A, B, C, then A again, then D. LRU evicts B, because A was used more recently. FIFO evicts A, because A was loaded first.

Exact LRU for 8 lines must remember the order of 8 blocks in every set, so real CPUs use cheaper rules that behave almost like LRU.

Run It: Counting the Lines per Set

`01-ways.c` places k blocks exactly 4096 bytes apart, so on the laptop's L1 (index bits 6 to 11) they all go to the same set. It reads them over and over in a cycle, each read waiting for the one before. The second column spaces them 4160 bytes apart, which puts each block in a different set:

```
$ ./01-ways              # Linux, AMD Ryzen 7 8845HS (L1: 8 lines per set)
 k   same set (4096 apart)   different sets (4160 apart)
 1         0.79 ns                0.79 ns
 8         0.79 ns                0.78 ns
 9         6.08 ns                0.79 ns
10         3.33 ns                0.78 ns
16         2.54 ns                0.79 ns
24         2.48 ns                0.80 ns
```

The rows left out follow the same pattern. Up to 8 blocks fit in the set, and every read hits L1 in 0.8 ns. With 9 blocks, every read misses L1 and goes to L2. Read in a cycle, 9 blocks in 8 lines is the worst case for LRU: the block it evicts is always the next one the loop reads. The same 9 blocks in different sets all hit, so these are conflict misses, not capacity misses. 24 blocks are only 1.5 KB.

Write Policies

Two choices decide what a cache does on a write.

On a write hit, the block is already in the cache:

- **Write-through:** update the cache's copy and also write the new value to the level below right away. The level below is always up to date, but every write costs a slow write.
- **Write-back:** update only the cache's copy, and set the line's **dirty bit** to record that the copy differs from the level below. When a dirty line is evicted, the cache writes the whole block back. Many writes to the same block cost one slow write.

On a write miss, the block is not in the cache:

- **Write-allocate:** fetch the block into the cache, then do the write as a write hit. This pays off when the program soon uses the block again.
- **No-write-allocate:** write straight to the level below and leave the cache unchanged.

Write-through usually goes with no-write-allocate, and write-back with write-allocate. The CPU caches of current laptops and servers are write-back and write-allocate, because the levels below are slow.

Measuring Cache Performance

- **Miss rate:** misses divided by accesses. The **hit rate** is 1 minus the miss rate.
- **Hit time:** the time to return data on a hit, about 4 cycles for L1.
- **Miss penalty:** the extra time a miss costs, for example about 500 cycles when the block comes from main memory.

The average time of one access is

```
average time = hit time + miss rate × miss penalty
```

Suppose every L1 miss goes to main memory. With a 99% hit rate, the average is 4 + 0.01 × 500 = 9 cycles. With a 97% hit rate it is 4 + 0.03 × 500 = 19 cycles. Two more misses per hundred reads make the program about twice as slow, so small differences in the miss rate matter.

What the Parameters Trade Off

Make this larger	What gets better	What it costs
cache size C	fewer capacity misses	a longer hit time, which is why L1 is kept small, 32 KB here
block size B	more bytes per miss for code with spatial locality	fewer lines in the same space, and more bytes to move on each miss
lines per set W	fewer conflict misses	more tags to compare on every read, which takes power and time

Real hierarchies split the difference: a small, fast L1 with 8 lines per set, then larger, slower levels. The laptop's L3 has 16 lines per set. A miss in L3 costs a trip to main memory, so there it is worth comparing more tags to avoid conflict misses.

Matrix Multiply: Three Loop Orders

`C = A × B` for n × n matrices of **`double`** can put its three loops in any order. Each order does the same n^3 multiply-adds. Only the inner loop matters, because it runs n^3 times:

```
// ijk: A by row, B by column
for (k = 0; k < n; k++)
    sum += A[i][k] * B[k][j];

// kij: B and C by row
for (j = 0; j < n; j++)
    C[i][j] += r * B[k][j];
```

```
// jki: A and C by column
for (i = 0; i < n; i++)
    C[i][j] += A[i][k] * r;
```

A 64-byte block holds 8 **`double`**s. Reading along a row misses once every 8 reads, 0.125 misses per iteration. Reading down a column of a large matrix misses every time, 1 miss per iteration:

Order	Inner loop reads	Predicted L1 misses per iteration
ijk	A by row, B by column	0.125 + 1 = 1.125
jki	A by column, C by column	1 + 1 = 2
kij	B by row, C by row	0.125 + 0.125 = 0.25

Run It: Matrix Multiply in Three Orders

`02-matmul.c` multiplies two 1000 × 1000 matrices of **`double`**, each 8 MB. **`perf stat`** reads the CPU's own event counters while a program runs; here it counts L1 misses:

```
$ perf stat -e L1-dcache-load-misses ./02-matmul jki   # Linux, Ryzen 7 8845HS
jki     0.84 s    0.84 ns per multiply-add
     2,011,760,430      L1-dcache-load-misses:u
```

Running all three orders, and dividing each count by the 10^9 multiply-adds:

Order	Time	Predicted misses per iteration	Measured by **`perf`**
ijk	0.58 s	1.125	1.14
jki	0.85 s	2	2.01
kij	0.23 s	0.25	0.13

kij misses less than predicted because both of its inner reads walk along a row, and the prefetcher fetches those blocks before the loop asks for them. With n = 1024 instead of 1000, ijk takes 5.84 s and jki 21.7 s, but kij still takes 0.27 s. Rows of 1024 **`double`**s are 8 KB long, so a column's elements have the same index bits and compete for a few sets.

Cache Coherence

Each core has its own L1 and L2 caches. If two cores have copies of the same block and one core writes to it, the other core's copy is out of date:

```
Core A's L1:  [X = 42]  <- core A just wrote X
Core B's L1:  [X = 0]   <- an old copy
```

The cores keep their caches **coherent**, meaning every core reads the most recent value written, with a protocol called **MESI**. Every line in every cache is in one of four states:

State	Meaning
Modified	Only this cache has the block, and it has been written (dirty)
Exclusive	Only this cache has the block, and it matches memory
Shared	Several caches have the block, and none has written it
Invalid	The line holds no usable data

Before a core writes a block, it tells the other caches to mark their copies Invalid. Their next read of that block misses and gets the new value. Coherence works on whole blocks, not on single variables.

Run It: False Sharing

A **thread** is a second flow of control inside the same process: it runs its own function, possibly on another core, and shares the process's memory. In **`03-falseshare.c`**, two threads each add 1 to their **own** counter 200 million times. First the two counters are 8 bytes apart, in one block. Then they are 64 bytes apart, in two blocks:

```
$ ./03-falseshare          # Linux, AMD Ryzen 7 8845HS
run 1: same block  0.21 s, different blocks  0.04 s,  5.0x
run 2: same block  0.20 s, different blocks  0.04 s,  4.9x
run 3: same block  0.21 s, different blocks  0.04 s,  4.9x
```

The threads never touch each other's counter, but every write by one core invalidates the other core's copy of the whole block. The block moves back and forth between the two cores. This is **false sharing**. The fix is to give each counter its own block:

```
struct { _Alignas(64) long a; _Alignas(64) long b; } counters;   // 128 bytes
```

AI Systems Connection

The attention step of a transformer compares every token of the input with every other token. For n tokens it computes an n × n matrix of scores, turns each row into weights that add up to 1 (a **softmax**), and multiplies the weights by another matrix.

Written step by step, each step writes the whole n × n matrix to the GPU's main memory and the next step reads it back. For n = 32,768 tokens, the matrix has about 10^9 entries, 2 GB at 2 bytes each, for every attention head. Most of the time goes to moving it, not to the arithmetic.

FlashAttention (Dao et al., 2022) never writes that matrix to memory. It splits the input into tiles that fit in the small, fast memory on the GPU chip. For each tile it computes the scores, applies them, and adds the result into the output. It keeps a running maximum and a running sum for each row, so the softmax comes out the same without the whole row at once. This is blocking, the same idea as for matrix multiplication, and **fusion**: three steps are done in one pass while the data is in fast memory. The results are the same, attention runs several times faster, and the memory it needs grows with n instead of n^2.

The Lecture in One Slide

- In a W-way set-associative cache, a block can go in any of the W lines of its set. More lines per set means fewer conflict misses.
- When a set is full, the cache evicts a block. LRU evicts the one used longest ago; real CPUs use rules close to LRU.
- Write-back caches write a block to the level below only when it is evicted, and the dirty bit records which lines need it.
- Average access time = hit time + miss rate × miss penalty, so a small change in the miss rate can double the time.
- Choose a loop order whose inner loop walks along rows. Measure with **`perf`**.
- Coherence works on whole blocks, so two threads writing different variables in one block slow each other down.

Practice

1. Take the 8 KB, 4-way cache with 64-byte blocks on the “Set-Associative Caches” slide. Which set does address **`0x9064`** go to, and what is its tag?
2. One set of a 2-way LRU cache starts empty. It reads blocks A, B, A, C, B, A. Which reads hit? Which block is evicted at each miss? Do the same with FIFO.
3. When would random eviction beat LRU? Use the measurement on “Run It: Counting the Lines per Set”.
4. A loop writes every **`int`** of a 4 KB array once; blocks are 64 bytes. With write-back and write-allocate, how many blocks are read from the level below, and how many are written back later? With write-through and no-write-allocate, how many writes reach the level below?
5. L1 hits take 4 cycles, and an L1 miss costs 12 more cycles because L2 has the block. What is the average access time at a 95% L1 hit rate? At 99%?
6. The other three matrix-multiply orders are ikj, jik and kji. For each, say what the inner loop reads and predict its L1 misses per iteration.
7. **`counters`** in **`03-falseshare.c`** starts a block. Would the threads still slow down using **`counters[0]`** and **`counters[4]`**? **`counters[16]`**?

Answers to Practice

1. S = 32, so the index is bits 6 to 10. 0x9064 is 36964; 36964 / 64 = 577, and 577 % 32 = 1, so set 1. The tag is 36964 / 2048 = 18.
2. LRU: A miss, B miss, A hit, C miss (evicts B, used longest ago), B miss (evicts A), A miss (evicts C). One hit. FIFO: A miss, B miss, A hit, C miss (evicts A, loaded first), B hit, A miss (evicts B). Two hits.
3. When a program reads, in a cycle, a few more blocks than one set has lines: 9 blocks in an 8-line set. LRU always evicts the block that is needed next, so every read misses. Random eviction keeps some of the blocks that are needed soon, so some reads hit.
4. The array is 64 blocks. Write-back with write-allocate: the first write to each block misses and reads the block in (64 reads); the other 15 writes hit. All 64 lines are dirty, so 64 blocks are written back. Write-through with no-write-allocate: every one of the 1024 writes goes to the level below, and no block is read.
5. 4 + 0.05 × 12 = 4.6 cycles at 95%; 4 + 0.01 × 12 = 4.12 cycles at 99%.
6. ikj is like kij: B and C by row, 0.25. jik is like ijk: A by row, B by column, 1.125. kji is like jki: A and C by column, 2.

Reading

- CS:APP §6.4.3–6.4.7 (Set-associative and fully associative caches, writes, performance) and §6.5–6.7 (Writing cache-friendly code, the matrix multiply example)
- The sample code in **`sample code/lecture16/`**